



Tarefa Semanal

Comparação sequencial x paralela com pthreads

Objetivo

Comparar experimentalmente uma implementação **sequencial** e uma implementação **paralela em C com pthreads**, analisando como o tamanho da entrada e o número de threads afetam o tempo de execução.

1. Organização dos grupos

- Grupos de **3 a 4 estudantes**.
- Cada grupo deve escolher **um** dos problemas da lista.
- Um integrante deve publicar nos **comentários da aula no Classroom** o problema escolhido. A escolha vale por ordem de registro para evitar repetição.
- O prazo e a forma de envio final serão os informados no Classroom.

2. O que deve ser implementado

Para o problema escolhido, o grupo deverá produzir:

1. uma versão **sequencial** em C;
2. uma versão **paralela** em C utilizando `pthread.h`;
3. verificação de que as duas versões produzem resultados equivalentes;
4. medição de tempo usando o mesmo critério nas duas versões.

3. Configurações obrigatórias

Para cada tamanho de entrada, execute:

Implementação	Configuração
Sequencial	1 fluxo de execução
Pthreads	2 threads
Pthreads	4 threads
Pthreads	8 threads
Pthreads	máximo de CPUs lógicas online do computador

O máximo do computador pode ser obtido em Linux com `sysconf(_SC_NPROCESSORS_ONLN)`. Caso o valor máximo coincida com 2, 4 ou 8, registre mesmo assim que aquela execução representa a capacidade lógica máxima observada na máquina.

Mais threads não significa melhor

A atividade não pretende provar que a versão paralela é sempre mais rápida. Entradas pequenas, excesso de threads, seções críticas grandes, cache e overhead podem fazer a versão paralela perder desempenho. Esse resultado deve ser discutido, não escondido.

4. Repetições e resultados

Cada combinação **entrada** × **configuração** deve ser executada pelo menos **3 vezes**. O relatório deve preservar os tempos individuais e calcular, no mínimo, a média.

Também calcule:

$$\text{Speedup } S_p = \frac{T_{seq}}{T_p} \quad \text{Eficiência } E_p = \frac{S_p}{p}.$$

É recomendável apresentar também desvio padrão.

5. Entradas fornecidas

A pasta `tarefa_semanal/entradas/` contém **48 arquivos**: três tamanhos para cada um dos 16 problemas. Os arquivos são determinísticos e podem ser recriados com `gerar_todas_entradas.py`.

Nº	Problema	Formato das entradas
1	Bucket Sort	Primeira linha: N . Depois, N inteiros.
2	Merge Sort	Primeira linha: N . Depois, N inteiros.
3	Quick Sort	Primeira linha: N . Depois, N inteiros.
4	Contagem/teste de primos	Primeira linha: N . Depois, N inteiros que devem ser testados.
5	Estatísticas de vetor	Primeira linha: N . Depois, N valores reais. Calcular soma, média, variância, mínimo e máximo.
6	Histograma	Primeira linha: N e número de classes. Depois, N inteiros representando as classes.
7	Convolução 1D	Primeira linha: N e tamanho do kernel. Segunda linha: kernel. Depois: sinal com N valores.
8	Produto escalar	Primeira linha: N . Depois, N linhas com um par $a_i b_i$.
9	Matriz-vetor	Primeira linha: ordem N . Depois, matriz $N \times N$ e, ao final, vetor com N valores.
10	Multiplicação de matrizes	Primeira linha: ordem N . Depois, duas matrizes quadradas $N \times N$.
11	Busca de padrão em texto	Cabeçalho informa o padrão e o número de linhas; depois vem o texto. Contar ocorrências ou linhas correspondentes.
12	Frequência de palavras	Primeira linha: quantidade aproximada de palavras; depois, tokens separados por espaços/quebras de linha.
13	Distâncias de pontos 3D	Primeira linha: N ; segunda: ponto de referência; depois, N pontos 3D.
14	Integração numérica	Arquivo informa número de subintervalos, limites e a função do experimento. Paralelize a soma numérica.
15	Soma/classificação por divisores	Primeira linha: N ; depois, inteiros. Para cada um, calcule soma/contagem de divisores conforme definido pelo grupo.

Nº	Problema	Formato das entradas
16	Média móvel	Primeira linha: N e largura da janela; depois, N valores reais.

6. Entendendo os problemas disponíveis

Antes de escolher o tema, leia a descrição abaixo. Em todos os casos, a proposta é construir **duas versões do mesmo problema**: uma sequencial e outra paralela com pthreads. A versão paralela deve dividir o trabalho entre as threads sem alterar o resultado final esperado.

Importante

Não basta simplesmente criar várias threads. O grupo deve identificar **qual parte do trabalho pode ser dividida**, como reunir os resultados parciais e onde pode existir acesso concorrente a dados compartilhados. Em alguns problemas quase não há necessidade de sincronização; em outros, mutexes ou outras estratégias podem ser necessários.

Problema 1 – Bucket Sort

O que é: algoritmo de ordenação que distribui os valores de entrada em diferentes “baldes” de acordo com suas faixas de valores. Cada balde é ordenado e, ao final, os conteúdos são concatenados para formar o vetor ordenado.

O que implementar: ler os N números, ordenar todos os valores e produzir a mesma sequência crescente nas versões sequencial e paralela.

Ideia de paralelização: diferentes threads podem processar partes do vetor, distribuir elementos em baldes ou ordenar baldes independentes. O grupo deverá tomar cuidado para que duas threads não modifiquem incorretamente a mesma estrutura compartilhada.

Problema 2 – Merge Sort

O que é: algoritmo de ordenação baseado em dividir o vetor em partes menores, ordenar essas partes e depois intercalá-las (*merge*) até obter o vetor completo ordenado.

O que implementar: ordenar os N inteiros e verificar que a saída da versão paralela é exatamente a mesma da versão sequencial.

Ideia de paralelização: as metades do vetor podem ser ordenadas simultaneamente porque, durante boa parte da execução, são independentes. Depois é necessário sincronizar o término dessas etapas antes de realizar a intercalação.

Problema 3 – Quick Sort

O que é: algoritmo de ordenação que escolhe um pivô, particiona os elementos entre valores menores e maiores que o pivô e repete o processo recursivamente nas partições.

O que implementar: ordenar corretamente todo o vetor para os três tamanhos de entrada.

Ideia de paralelização: depois de uma partição, as regiões esquerda e direita podem ser processadas de maneira independente. O desafio é evitar criar threads demais para partições muito pequenas, pois o custo de criação das threads pode superar o ganho obtido.

Problema 4 – Contagem/teste de números primos

O que é: para cada número da entrada, determinar se ele é primo, isto é, se possui exatamente dois divisores positivos: 1 e ele próprio.

O que implementar: testar todos os números e informar, por exemplo, quantos são primos e/ou quais valores são primos.

Ideia de paralelização: dividir o conjunto de números entre as threads. Como o teste de um número não depende do teste de outro, esse problema permite uma divisão simples do trabalho. Ao final, os resultados parciais precisam ser combinados corretamente.

Problema 5 – Estatísticas de um vetor

O que é: calcular medidas descritivas de um conjunto de valores: soma, média, variância, mínimo e máximo.

O que implementar: ler todos os valores reais e produzir as estatísticas solicitadas com o mesmo resultado nas duas versões.

Ideia de paralelização: cada thread recebe um trecho do vetor e calcula resultados parciais, como soma parcial, mínimo local e máximo local. Depois, a thread principal combina essas informações para obter o resultado global. A variância pode exigir uma segunda passagem ou uma estratégia equivalente.

Problema 6 – Histograma

O que é: contar quantos elementos pertencem a cada classe ou categoria. O resultado é um vetor de frequências.

O que implementar: percorrer os N valores e informar a quantidade de ocorrências de cada classe.

Ideia de paralelização: cada thread pode analisar uma parte da entrada. O principal ponto de atenção é a atualização das contagens: se várias threads incrementarem diretamente o mesmo contador, pode ocorrer condição de corrida. Uma alternativa é usar histogramas locais e combiná-los ao final.

Problema 7 – Convolução 1D

O que é: aplicar um pequeno vetor chamado *kernel* sobre um sinal maior. Cada posição da saída é calculada combinando uma vizinhança do sinal com os pesos do kernel.

O que implementar: gerar o vetor resultante da convolução para o sinal e o kernel fornecidos.

Ideia de paralelização: diferentes posições da saída podem ser calculadas por threads diferentes, pois cada posição utiliza apenas os valores necessários do sinal de entrada. É necessário definir claramente como serão tratadas as bordas do vetor.

Problema 8 – Produto escalar

O que é: dados dois vetores A e B , calcular $A \cdot B = \sum_i a_i b_i$.

O que implementar: multiplicar os pares correspondentes e somar todos os produtos, gerando um único valor final.

Ideia de paralelização: cada thread calcula a soma dos produtos de uma faixa dos vetores. Ao final, as somas parciais são reduzidas a um único resultado. Atualizar uma soma global a cada elemento deve ser evitado, pois criaria muita contenção.

Problema 9 – Multiplicação matriz-vetor

O que é: multiplicar uma matriz quadrada A por um vetor x , produzindo um vetor y . Cada posição de y corresponde ao produto escalar de uma linha da matriz com o vetor.

O que implementar: calcular corretamente todas as posições do vetor resultante.

Ideia de paralelização: distribuir linhas da matriz entre as threads. Como cada linha produz uma posição independente no vetor de saída, as threads podem trabalhar simultaneamente com pouca necessidade de sincronização.

Problema 10 – Multiplicação de matrizes

O que é: dadas duas matrizes quadradas A e B , calcular a matriz $C = A \times B$. Cada elemento c_{ij} é obtido multiplicando uma linha de A por uma coluna de B .

O que implementar: produzir toda a matriz resultado e validar que as versões sequencial e paralela geram os mesmos elementos.

Ideia de paralelização: linhas, blocos ou conjuntos de células da matriz resultante podem ser distribuídos entre as threads. Para entradas grandes, o grupo também deve observar o impacto de cache e acesso à memória.

Problema 11 – Busca de padrão em texto

O que é: procurar uma palavra ou sequência de caracteres em um texto e contar quantas vezes ela aparece ou em quantas linhas ela ocorre.

O que implementar: percorrer o texto fornecido e retornar uma contagem bem definida das ocorrências do padrão.

Ideia de paralelização: dividir linhas ou blocos do texto entre as threads. Cada thread realiza buscas locais e produz uma contagem parcial. Se o grupo dividir o texto por posições arbitrárias, deve tomar cuidado com padrões que atravessem a fronteira entre dois blocos.

Problema 12 – Frequência de palavras

O que é: contar quantas vezes cada palavra aparece em um conjunto de tokens, semelhante a uma tabela de frequências de palavras.

O que implementar: produzir uma lista ou estrutura contendo cada palavra distinta e sua quantidade de ocorrências.

Ideia de paralelização: cada thread pode processar uma parte dos tokens e construir contagens locais. Depois, essas tabelas precisam ser combinadas. Se todas as threads atualizarem diretamente uma única tabela global, haverá necessidade de sincronização e possivelmente maior contenção.

Problema 13 – Distâncias de pontos 3D

O que é: calcular a distância euclidiana entre um ponto de referência (x_r, y_r, z_r) e cada ponto (x_i, y_i, z_i) da entrada:

$$d_i = \sqrt{(x_i - x_r)^2 + (y_i - y_r)^2 + (z_i - z_r)^2}.$$

O que implementar: calcular uma distância para cada ponto e, opcionalmente, identificar o ponto mais próximo e o mais distante.

Ideia de paralelização: dividir os pontos entre as threads. Cada distância é independente. Se forem procurados mínimo e máximo globais, cada thread pode produzir extremos locais para combinação posterior.

Problema 14 – Integração numérica

O que é: aproximar a área sob uma função em um intervalo usando muitos pequenos subintervalos, por exemplo pelo método dos retângulos ou dos trapézios.

O que implementar: calcular uma aproximação numérica da integral da função indicada no arquivo de entrada utilizando o número de subintervalos fornecido.

Ideia de paralelização: dividir os subintervalos entre as threads. Cada thread calcula uma soma parcial da área e, no final, essas somas são combinadas. É um exemplo clássico de redução paralela.

Problema 15 – Soma/classificação por divisores

O que é: analisar cada inteiro da entrada calculando propriedades relacionadas aos seus divisores, como quantidade de divisores ou soma dos divisores. O grupo deverá declarar claramente qual propriedade será usada.

O que implementar: para cada número, calcular a propriedade escolhida e apresentar os resultados de maneira verificável. É permitido também classificar os números a partir desse valor, desde que o critério seja documentado.

Ideia de paralelização: distribuir os números entre as threads, já que a análise de um número é independente da análise dos demais. Entradas contendo números maiores tendem a aumentar o custo do teste de divisores e podem tornar o ganho de paralelismo mais perceptível.

Problema 16 – Média móvel

O que é: para uma sequência numérica e uma janela de largura W , calcular para cada posição a média dos elementos pertencentes àquela janela. Esse tipo de operação é comum em séries temporais e suavização de sinais.

O que implementar: produzir a sequência de médias móveis de acordo com a largura de janela informada no arquivo.

Ideia de paralelização: diferentes posições da saída podem ser atribuídas a threads diferentes. As threads apenas leem o vetor original e escrevem em posições distintas da saída, o que permite paralelização com pouca sincronização. O grupo deve definir corretamente as posições em que a janela é válida.

Critério de escolha

Escolha um problema que o grupo consiga explicar e implementar. O objetivo não é selecionar o algoritmo “mais difícil”, mas realizar uma comparação experimental correta entre a versão sequencial e a paralela e justificar os resultados obtidos.

7. Gráficos obrigatórios

O relatório deverá conter, pelo menos:

1. gráfico de **tempo médio** comparando sequencial, 2, 4, 8 e máximo de threads para as três entradas;
2. gráfico de **speedup**;

3. interpretação textual explicando onde houve ganho, saturação ou piora.

8. Entregáveis

- códigos-fonte .c;
- instruções de compilação e execução;
- arquivos de resultados brutos (CSV ou TXT);
- gráficos;
- relatório curto contendo máquina utilizada, compilador/opções, metodologia, resultados e análise;
- identificação dos integrantes e do problema escolhido.

9. Checklist de metodologia

- Use a **mesma entrada** nas duas versões.
- Meça exatamente a **mesma região** do algoritmo.
- Não inclua leitura de arquivo em uma versão e exclua na outra.
- Compile ambas com a mesma política de otimização, por exemplo -O2 ou -O3.
- Não altere o algoritmo sequencial apenas para fazê-lo artificialmente pior.
- Registre o número de CPUs lógicas e as características básicas do computador.
- Preserve resultados que contrariem a hipótese inicial.